```
# Heart Disease Prediction with an Artificial Neural Network (ANN)

## Objective
Predict whether a patient has heart disease (`1`) or not (`0`) from basic clinical
information, using a feedforward neural network built with TensorFlow/Keras.
This is a binary classification problem.

## Dataset
- **File:** `heart_disease.csv`
- **Size:** 200 records, 7 columns, no missing values
- **Features:**
  - `age`: patient age
  - `sex`: patient sex (encoded as 0/1)
  - `cholesterol`: cholesterol level
  - `blood_pressure`: blood pressure reading
  - `max_heart_rate`: maximum heart rate achieved
  - `exercise_angina`: exercise-induced angina (0 = no, 1 = yes)
- **Target:** `heart_disease` (0 = no disease, 1 = disease)

## Methodology
1. Loaded the data and checked its shape, data types, and missing values
2. Encoded `sex` with `LabelEncoder`
3. Ran `SelectKBest` (ANOVA F-test) to rank feature importance
4. Split the data 80/20 into training and test sets (`random_state=42`)
5. Standardized the features with `StandardScaler`
6. Built a Sequential ANN:
   - Dense(32, ReLU) → Dense(16, ReLU) → Dense(1, Sigmoid)
   - Optimizer: Adam, loss: binary cross-entropy
7. Trained for up to 500 epochs with a 20% validation split and `EarlyStopping`
   (patience = 10, restoring best weights)
8. Evaluated on the test set with accuracy, loss, and a classification report

## Results
- Test accuracy: **~60%**
- Test loss: **~0.695**

The model performs only modestly above chance. The dataset is small (200 rows),
so there is limited data for a neural network to learn from.

## Limitations and Future Work
- Try the full dataset size or a larger dataset
- Compare against simpler baselines (Logistic Regression, Random Forest)
- Tune hyperparameters (layers, neurons, learning rate, dropout)
- Use cross-validation for more reliable performance estimates
- Report confusion matrix, recall, and ROC-AUC, since missing a disease case matters more than a false alarm

## Tech Stack
Python, Pandas, Seaborn, Matplotlib, Scikit-learn, TensorFlow/Keras

## Disclaimer
This is a learning project and is not a medical diagnostic tool.
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("heart_disease.csv")
df.head()
```

	age	sex	cholesterol	blood_pressure	max_heart_rate	exercise_angina	heart_disease
0	67	0	184	99	154	0	0
1	57	1	198	162	192	1	1
2	43	0	166	113	161	0	0
3	71	1	307	153	62	0	1
4	36	0	195	138	82	0	0

```
df.shape
```

```
(200, 7)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   age              200 non-null    int64
 1   sex              200 non-null    int64
 2   cholesterol      200 non-null    int64
 3   blood_pressure   200 non-null    int64
 4   max_heart_rate   200 non-null    int64
 5   exercise_angina  200 non-null    int64
 6   heart_disease    200 non-null    int64
dtypes: int64(7)
memory usage: 11.1 KB
```

```
df.isnull().sum()
```

	0
age	0
sex	0
cholesterol	0
blood_pressure	0
max_heart_rate	0
exercise_angina	0
heart_disease	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["sex"] = le.fit_transform(df["sex"])
```

```
df.head()
```

	age	sex	cholesterol	blood_pressure	max_heart_rate	exercise_angina	heart_disease
0	67	0	184	99	154	0	0
1	57	1	198	162	192	1	1
2	43	0	166	113	161	0	0
3	71	1	307	153	62	0	1
4	36	0	195	138	82	0	0

```
df.columns
```

```
Index(['age', 'sex', 'cholesterol', 'blood_pressure', 'max_heart_rate',
       'exercise_angina', 'heart_disease'],
      dtype='object')
```

```
# Setting the target and Feature Variables
```

```
x = df[['age', 'sex', 'cholesterol', 'blood_pressure', 'max_heart_rate',
       'exercise_angina']]
y = df['heart_disease']
```

```
from sklearn.feature_selection import SelectKBest,f_classif
```

```
skb = SelectKBest(f_classif, k =5)
select = skb.fit_transform(x,y)
column_name = x.columns[skb.get_support()]
column_name
```

```
Index(['age', 'sex', 'cholesterol', 'max_heart_rate', 'exercise_angina'], dtype='object')
```

```
x_new = ['age', 'sex', 'cholesterol', 'max_heart_rate', 'exercise_angina']
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size = 0.2, random_state = 42)
xtrain.shape, xtest.shape, ytrain.shape, ytest.shape
```

```
((160, 6), (40, 6), (160,), (40,))
```

```
from sklearn.preprocessing import StandardScaler
```

```
SS = StandardScaler()
xtrain = SS.fit_transform(xtrain)
xtest = SS.fit_transform(xtest)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()

model.add(Dense(32, activation = "relu", input_shape = (xtrain.shape[1],)))
model.add(Dense(16, activation = "relu"))
model.add(Dense(1, activation = "sigmoid"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compiling the model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Training the model
history = model.fit(
    xtrain, ytrain,
    batch_size=32,
    epochs=500,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

```
Epoch 1/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 1s 111ms/step - accuracy: 0.5234 - loss: 0.6971 - val_accuracy: 0.6250 - val_loss: 0.6754
Epoch 2/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - accuracy: 0.5391 - loss: 0.6912 - val_accuracy: 0.6250 - val_loss: 0.6749
Epoch 3/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.5469 - loss: 0.6867 - val_accuracy: 0.6250 - val_loss: 0.6751
Epoch 4/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step - accuracy: 0.5391 - loss: 0.6829 - val_accuracy: 0.5938 - val_loss: 0.6755
Epoch 5/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.5625 - loss: 0.6790 - val_accuracy: 0.5625 - val_loss: 0.6767
Epoch 6/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.5703 - loss: 0.6755 - val_accuracy: 0.5625 - val_loss: 0.6780
Epoch 7/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 0.5703 - loss: 0.6729 - val_accuracy: 0.5000 - val_loss: 0.6800
Epoch 8/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step - accuracy: 0.6094 - loss: 0.6696 - val_accuracy: 0.5312 - val_loss: 0.6816
Epoch 9/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.6328 - loss: 0.6671 - val_accuracy: 0.5625 - val_loss: 0.6835
Epoch 10/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 0.6250 - loss: 0.6644 - val_accuracy: 0.5312 - val_loss: 0.6852
Epoch 11/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - accuracy: 0.6406 - loss: 0.6617 - val_accuracy: 0.5000 - val_loss: 0.6869
Epoch 12/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.6797 - loss: 0.6589 - val_accuracy: 0.4375 - val_loss: 0.6889
```

```
# Evaluating the model
loss, accuracy = model.evaluate(xtest, ytest)
```

```
print("Accuracy", accuracy)
print("Loss", loss)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 58ms/step - accuracy: 0.5250 - loss: 0.6978
Accuracy 0.5249999761581421
Loss 0.6978481411933899
```

```
# Making predictions
y_pred = model.predict(xtest)
y_pred[0:5]
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 327ms/step
array([[0.44388554],
       [0.45379883],
       [0.4746941 ],
       [0.47834554],
       [0.42975894]], dtype=float32)
```

```
# Converting from Probability
y_pred = (y_pred > 0.5).astype(int)
y_pred[0:5]
```

```
array([[0],
       [0],
       [0],
       [0],
       [0]])
```

```
y_compare = pd.DataFrame(ytest)
y_compare['Predicted'] = y_pred
y_compare = y_compare.reset_index()
y_compare.drop('index', axis = 1)
```

	heart_disease	Predicted
0	1	0
1	1	0
2	0	0
3	1	0
5	0	0
7	0	1
8	0	0
9	1	0
10	0	0
11	1	0
12	0	0
15	0	0
16	0	0
17	0	0
18	1	0
19	1	0
20	0	0
21	0	0
22	1	0
23	1	0
24	1	0
25	0	0
26	1	0

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
cr = classification_report(ytest, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.54      0.91      0.68        22
           1       0.33      0.06      0.10        18

    accuracy                           0.53        40
   macro avg       0.44      0.48      0.39        40
weighted avg       0.45      0.53      0.42        40
```